

Week-5 Practical

1. Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

int main() {
    int pipefd[2];
    pid_t pid;
    int data[5] = {10, 20, 30, 40, 50};

    if (pipe(pipefd) == -1) {
        perror("pipe");
        return 1;
    }

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid > 0) {
        // Parent = Producer
        close(pipefd[0]);

        clock_t start = clock();

        printf("Producer: Sending data...\n");
        write(pipefd[1], data, sizeof(data));

        printf("Producer: Data sent successfully.\n");
        close(pipefd[1]);

        wait(NULL);

        clock_t end = clock();

        double time_taken =
            ((double)(end - start)) / CLOCKS_PER_SEC;

        printf("Communication time: %f seconds\n", time_taken);
    }
    else {
        // Child = Consumer
        close(pipefd[1]);

        int received[5];

        read(pipefd[0], received, sizeof(received));

        printf("Consumer: Received data:\n");

        for (int i = 0; i < 5; i++) {
            printf("%d ", received[i]);
        }

        printf("\n");

        close(pipefd[0]);

        exit(0);
    }

    return 0;
}
```

Output:

```
priyanka@DESKTOP-H0ISO7A:~$ cd ~
mkdir -p OS_Practical/Week-5/src OS_Practical/Week-5/bin
cd OS_Practical/Week-5
priyanka@DESKTOP-H0ISO7A:~/OS_Practical/Week-5$ pwd
/home/priyanka/OS_Practical/Week-5
priyanka@DESKTOP-H0ISO7A:~/OS_Practical/Week-5$ nano src/producer_consumer.c

priyanka@DESKTOP-H0ISO7A:~/OS_Practical/Week-5$ gcc src/producer_consumer.c -o bin/producer_consumer
priyanka@DESKTOP-H0ISO7A:~/OS_Practical/Week-5$ ./bin/producer_consumer
Producer: Sending data...
Producer: Data sent successfully.
Consumer: Received data:
10 20 30 40 50
Communication time: 0.000146 seconds
```

2. Develop a program that executes the equivalent of the shell command: `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

```
priyanka@DESKTOP-HOISO7A: ~$ nano src/ls_grep.c
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pipefd[2];
    pid_t pid1, pid2;

    if (pipe(pipefd) == -1) {
        perror("pipe");
        return 1;
    }

    // Create first child for ls -l
    pid1 = fork();

    if (pid1 < 0) {
        perror("fork");
        return 1;
    }

    if (pid1 == 0) {
        // Child 1: execute ls -l

        close(pipefd[0]);

        dup2(pipefd[1], STDOUT_FILENO);

        close(pipefd[1]);

        execlp("ls", "ls", "-l", NULL);

        perror("execlp ls");
        exit(1);
    }

    // Create second child for grep ".c"
    pid2 = fork();

    if (pid2 < 0) {
        perror("fork");
        return 1;
    }
}
```

Output:

```
priyanka@DESKTOP-HOISO7A:~/OS_Practical/Week-5$ gcc src/ls_grep.c -o bin/ls_grep
priyanka@DESKTOP-HOISO7A:~/OS_Practical/Week-5$ ./bin/ls_grep
drwxr-xr-x 2 priyanka priyanka 4096 Sep 10 10:18 src
Command execution completed.
```